Introduction to Greedy Algorithms

What is a Greedy Algorithm?

Definition. *A **greedy algorithm** is a method used to solve optimization problems. It builds the solution step-by-step, always choosing the best available option at each step—this is known as making a locally optimal choice.*
*Unlike **dynamic programming**, greedy algorithms **do not reconsider previous decisions**.*

Key Characteristics

- **Greedy Choice Property**: Always picks the most promising option at the current step.
- **Optimal Substructure**: The optimal solution to the problem includes optimal solutions to its subproblems.
- **Local Optimality**: Makes choices that seem best in the moment, aiming for a globally optimal solution.
- **Irrevocable Decisions**: Once a choice is made, it isn't changed—this leads to faster performance, but not always the best result.

Common Examples

- **Huffman Coding** – for efficient data compression.
- **Prim's Algorithm** – builds a minimum spanning tree from a graph.
- **Kruskal's Algorithm** – another way to find a minimum spanning tree, using edge sorting.
- **Dijkstra's Algorithm** – finds the shortest path in a weighted graph (note: greedy, but only works with non-negative edge weights).

Huffman Coding (Simplified Explanation)

What is Huffman Coding?

Huffman coding is a method used to compress data without losing any information. It works by using **shorter codes for common characters** and **longer codes for rare ones**, which helps reduce the total size of the data.

How It Works

1. **Count Frequencies** First, count how often each character appears in the data.
2. **Make a Min-Heap (Priority Queue)**
   Put all characters into a min-heap based on how often they appear. Characters that appear less often are given higher priority.
3. **Build the Huffman Tree**
   - Take the two characters with the smallest frequencies.
   - Combine them into a new node.
   - The new node's frequency is the sum of the two.
   - Repeat until only one node is left — this becomes the Huffman Tree.
4. **Assign Binary Codes**
   Go through the tree and assign codes:
   - Going left adds a "0"
   - Going right adds a "1"
5. **Encode the Data**
   Replace each character in your data with its binary code.

Example

Given the following character frequencies:

Character	Frequency
A	5
B	9
C	12
D	13
E	16
F	45

Steps:

- Initial heap: (A,5), (B,9), (C,12), (D,13), (E,16), (F,45)
- Merge A + B $\rightarrow$ (14)
- Merge C + D $\rightarrow$ (25)
- Merge AB + E $\rightarrow$ (30)
- Merge CD + ABE $\rightarrow$ (55)

- Merge F + ABCDE → (100)

Now we have a Huffman Tree. **Possible binary codes:**

- A: 1100
- B: 1101
- C: 100
- D: 101
- E: 111
- F: 0

Example encoding:
The word `ABCDEF` becomes: `11001101100101110`

Why It's Called Greedy

Huffman coding is a **greedy algorithm** because at each step, it **chooses the two smallest frequencies** to combine. This local best choice results in an overall optimal and efficient encoding.

Introduction to Prim's Algorithm

Prim's Algorithm is a **greedy algorithm** used to find a **Minimum Spanning Tree (MST)** for a **connected**, **undirected** graph with **weighted edges**. An MST connects all the vertices with the **minimum possible total edge weight**, without forming any cycles.

History

- First discovered by **Vojtěch Jarník** in 1930.
- Independently rediscovered by **Robert C. Prim** in 1957.

Steps of Prim's Algorithm

Step 1. Initialization:

- Start from an arbitrary vertex (source).
- Mark this vertex as part of the MST.
- Add all edges connected to this vertex into a **priority queue** (or min-heap), sorted by edge weight.

Step 2. Choose the Minimum Edge:

- Select the edge with the smallest weight from the priority queue.
- Ensure it connects to a vertex not yet in the MST.

Step 3. Add to MST:

- Add the selected edge and the connected vertex to the MST.
- Mark the newly connected vertex as part of the MST.

Step 4. Repeat:

- Add all edges from the newly connected vertex to the priority queue.
- Skip any edge that connects to a vertex already in the MST.
- Repeat until all vertices are included in the MST.

Step 5. Termination:

- The algorithm stops when all vertices are included in the MST.

Kruskal's Algorithm - Minimum Spanning Tree (MST)

Purpose

Kruskal's Algorithm is used to find the **Minimum Spanning Tree (MST)** of a connected, weighted graph. It minimizes the total edge weight without forming cycles.

Steps

1. **Sort the Edges**: Arrange all edges in non-decreasing order of weight.
2. **Initialize MST**: Start with an empty set to store MST edges.
3. **Use Union-Find (Disjoint Set Union - DSU)**:
   - `Find(u)`: Find the set/component containing vertex u.
   - `Union(u, v)`: Merge the sets containing u and v.
4. **Add Edges**:
   - For each edge (u, v) in sorted order:
     - If `Find(u)` $\neq$ `Find(v)`:
       - Add (u, v) to MST.
       - Perform `Union(u, v)`.
5. **Stop**: When MST has $(V - 1)$ edges.

Time Complexity

- Sorting edges: $O(E \log E)$
- Union-Find operations: $O(E \cdot \alpha(V))$
- Overall: $\mathbf{O(E \log E)}$

Example

Edges and weights:

$$(A, B, 1),\ (A, C, 3),\ (B, C, 2),\ (B, D, 4),\ (C, D, 5)$$

Sorted edges:

$$(A, B),\ (B, C),\ (A, C),\ (B, D),\ (C, D)$$

MST Construction:

- Add (A, B)
- Add (B, C)
- Skip (A, C) (forms a cycle)
- Add (B, D)
- Skip (C, D) (forms a cycle)

Final MST:

$$\{(A, B), (B, C), (B, D)\}, \quad \text{Total weight} = 7$$

Applications

- Network Design (cables, roads, pipelines)
- Clustering algorithms
- Image segmentation
- Communication networks

Introduction to Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from a source vertex to all other vertices in a graph with non-negative edge weights. It is widely used in routing and navigation systems.

How It Works

The algorithm maintains a set of vertices whose shortest distance from the source is known and repeatedly selects the vertex with the smallest known distance. The steps are as follows:

1. Assign a tentative distance value to every node: set it to zero for the initial node and infinity for all other nodes.
2. Set the initial node as current. Mark all other nodes as unvisited.
3. For the current node, consider all of its unvisited neighbors and calculate their tentative distances through the current node. Compare the newly calculated tentative distance with the current assigned value and assign the smaller one.
4. Once all neighbors of the current node are considered, mark the current node as visited. A visited node will not be checked again.
5. If the destination node has been marked visited or if the smallest tentative distance among the unvisited nodes is infinity, then stop. The algorithm has finished.
6. Otherwise, select the unvisited node with the smallest tentative distance and set it as the new current node. Go back to step 3.